GreenMind AI: An Explainable Multi-Agent Framework for Sustainable Cloud Resource Optimization Using Machine Learning and Digital Twin Simulation

Anonymous Submission Draft | [Author 1], [Author 2], [Author 3]
[Institution Name], [City, Country] | [contact@institution.edu]

Abstract

Modern cloud operations require simultaneous optimization of resource utilization, infrastructure cost, and environmental footprint. We present GreenMind AI, an open-source decision-support prototype integrating five one-hour-ahead gradient-boosted machine learning forecasters, a five-domain multi-agent analysis engine (Cost, Performance, Sustainability, Security, Reliability), deterministic conflict arbitration, a non-destructive Digital Twin simulator, and an eight-tool explainable AI copilot. The platform is built on FastAPI (Python 3.12) with a TypeScript/React frontend and cloud connectors for AWS, Azure, and GCP.

Evaluation uses a deterministic 60-day synthetic dataset (5,756 aligned records; strict 80/20 chronological holdout: N_train=4,604, N_test=1,152). GradientBoostingRegressor models achieve MAE improvements of 25.4%-45.4% over a one-hour forecast-origin persistence baseline. A technical audit reveals the stored rolling-mean baseline evaluates a 15-minute prediction horizon rather than the target one-hour horizon; the corrected four-step rolling baseline yields higher error than persistence for CPU (7.263% vs. 6.500%) due to diurnal phase shift. We document synthetic data provenance, unverified live connectors, a carbon-intensity scale mismatch in the recursive forecasting wrapper, and the HTTP 501 safety guard preventing live cloud mutations. The architecture is sound for decision support; production deployment requires evaluation on real cluster traces and live integration testing.

Keywords: cloud computing; workload forecasting; gradient boosting; carbon-aware computing; multi-agent systems; digital twin; explainability; GreenOps; FinOps; decision support.

1. Introduction

Global cloud computing infrastructure consumed an estimated 200-250 TWh of electricity in 2023, representing approximately 1% of global electricity demand, with projections indicating continued growth driven by AI workloads and data services [4, 6]. Cloud providers and enterprise tenants are responding with GreenOps disciplines - analogous to FinOps financial governance - seeking to minimize both cost and carbon emissions through workload scheduling, right-sizing, and regional migration to cleaner energy grids.

Proactive workload forecasting enables orchestrators to anticipate resource bottlenecks, defer batch workloads to low-carbon grid hours, and plan right-sizing actions in advance of demand spikes. However, high predictive accuracy on historical data does not inherently translate into realized financial savings, carbon abatement, or SLO preservation in production systems [3, 8]. Key barriers include: (i) synthetic datasets that do not capture real-world workload burstiness; (ii) conflicting multi-objective recommendations from specialized optimization agents; (iii) absence of safe, non-destructive mechanisms to preview infrastructure changes before live application.

GreenMind AI is an open-source prototype addressing these barriers by integrating a multi-metric forecasting pipeline, FastAPI service layer, TypeScript/React management cockpit, five domain-oriented analysis agents orchestrated via a LangGraph state graph, deterministic conflict resolution, an eight-tool explainable copilot, and a non-destructive Digital Twin simulator. Cloud connectors exist for AWS CloudWatch, Azure Monitor, and GCP Cloud Monitoring. Live remediation is intentionally restricted (HTTP 501), making it a decision-support rather than autonomous control system.

Research Contributions:

1. A detailed architectural description of GreenMind AI's six-component pipeline: telemetry ingestion, multi-metric gradient boosted forecasting, multi-agent domain analysis, deterministic conflict resolution, Digital Twin simulation, and copilot decision support.
2. Reproduction and verification of held-out forecasting performance across five targets (CPU, memory, network, cost, carbon proxy) with baseline alignment audit correcting the stored 15-min vs. 1-hour horizon discrepancy.
3. Explicit delineation of empirical evidence boundaries: synthetic data provenance, unverified live connectors, a carbon scale mismatch in the recursive forecasting wrapper, and the HTTP 501 safety barrier.

2. Background and Related Work

2.1 Cloud Workload Forecasting

Resource demand forecasting for cloud infrastructure has evolved from classical statistical models to modern deep learning. Calheiros et al. [7] demonstrated proactive cloud provisioning using ARIMA models, showing predictive scaling reduces QoS violations versus reactive threshold policies. Friedman's Gradient Boosting Machine [1] constructs additive tree models through functional gradient descent suited to structured tabular lag features without the computational scale of recurrent architectures. Rossi et al. [3] evaluated probabilistic LSTMs and Bayesian neural networks on Google and Alibaba cluster traces, showing point forecasts fail to capture operational uncertainty and accuracy degrades under cross-provider transfer learning.

Bergmeir, Hyndman, and Koo [2] provided theoretical grounding for time-series cross-validation, establishing that chronological holdout remains standard practice when simulating prospective deployment to avoid forward-looking data leakage. GreenMind AI enforces a strict 80/20 chronological split with no observation shuffling.

2.2 Carbon-Aware Computing and Green Algorithms

Henderson et al. [4] established systematic reporting standards for ML energy consumption and grid carbon footprints, defining computational carbon accounting components: TDP, PUE, runtime, and grid intensity. Lannelongue, Grealey, and Inouye [5] formulated the Green Algorithms framework, standardizing carbon footprint quantification using power draw, memory, runtime, and regional grid intensity. Both frameworks highlight that meaningful carbon accounting requires real-time hardware telemetry rather than proxy estimates.

Radovanovic et al. [6] deployed carbon-aware computing at Google datacenters using day-ahead grid predictions and Virtual Capacity Curves (VCCs) to temporally shift flexible batch workloads to cleaner grid hours, demonstrating significant carbon reduction without service degradation. Dodge et al. [8] measured operational carbon intensity across cloud instance types at AWS, GCP, and Azure, showing region selection and temporal scheduling are primary levers for cloud Scope 2 emission reduction.

2.3 Multi-Agent Systems and Digital Twins for Cloud Management

Multi-agent systems decompose complex multi-objective optimization into specialized agents negotiating through structured communication. Digital Twin technology applies virtual replicas of physical systems to evaluate proposed changes in simulation before live application - a critical risk management tool for production cloud infrastructure. GreenMind AI combines LangGraph-orchestrated agents and a state-snapshot Digital Twin engine for safe, explainable decision support.

3. System Architecture and Implementation

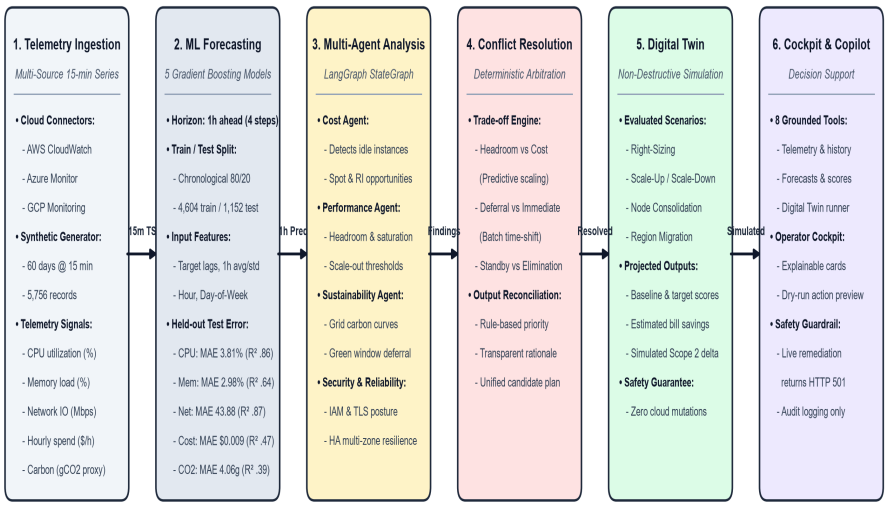


Fig. 1. End-to-end GreenMind AI pipeline architecture: multi-source telemetry ingestion -> ML multi-metric forecasting -> specialized multi-agent analysis -> deterministic conflict resolution -> Digital Twin simulation -> operator decision support.

Figure 1. GreenMind AI end-to-end pipeline: multi-source telemetry ingestion -> multi-metric ML forecasting -> specialized multi-agent analysis -> deterministic conflict resolution -> Digital Twin simulation -> operator decision support via AI copilot.

3.1 Telemetry Ingestion and Service Layer

The backend is implemented in Python 3.12 using FastAPI, with SQLite for development and PostgreSQL for production. The system ingests five continuous telemetry signals at 15-minute intervals: CPU utilization (%), memory load (%), network traffic (Mbps), hourly compute cost (USD/h), and a synthetic carbon emissions proxy (gCO2/15-min). Telemetry is maintained in a ring buffer for sliding-window historical queries.

Three cloud connector modules interface with major hyperscale providers: AWS CloudWatch via boto3, Azure Monitor REST APIs, and GCP Cloud Monitoring v3. Each implements a unified provider interface

and falls back to the deterministic synthetic generator transparently when credentials are absent. User credentials are scoped per session and encrypted in-memory via credential_crypto.py.

3.2 Multi-Metric Machine Learning Predictor

Five independent GradientBoostingRegressor models (scikit-learn) forecast each target metric one hour ahead (delta=4 steps x 15 min). Shared hyperparameters: 150 estimators, max depth 3, learning rate 0.08, random_state=42. Training completes in approximately 6.4 seconds on a standard development laptop.

Feature construction is strictly backward-looking to prevent temporal data leakage. CPU, memory, and network models use: (1) current value y_t; (2) hour of day, day of week; (3) one-hour rolling mean mu_{t,4}=(1/4)*sum(y_{t-i}, i=0..3); (4) rolling sigma_{t,4}. Cost model: current CPU, memory, cost, hour, day. Carbon model: regional grid intensity (gCO2/kWh), hour, day, CPU. Anomaly detection flags z-score > 2.5. Model artifacts serialized via joblib and loaded lazily on first inference.

3.3 Multi-Agent Analysis Engine

Five specialized agents execute as a LangGraph StateGraph:

- CostAgent: Evaluates compute spend rates, detects underutilized instances (CPU < 10%), recommends right-sizing, Spot instance conversion, or 1-year Reserved Instance commitments.
- PerformanceAgent: Monitors CPU/memory saturation and network bandwidth. Triggers predictive scale-out when saturation is projected to exceed 75%.
- SustainabilityAgent: Inspects 24-hour regional grid carbon-intensity curves. Recommends deferring non-urgent batch workloads to green windows (01:00-06:00 UTC).
- SecurityAgent: Audits IAM least-privilege compliance, TLS enforcement, EBS encryption, and open security group ports (SSH/RDP to 0.0.0.0/0).
- ReliabilityAgent: Evaluates multi-AZ topology, standby headroom, automated backup schedules (RPO compliance), and ALB health check status.

When LangGraph is unavailable, agents execute sequentially with recommendation ID deduplication (highest-priority wins). Each execution step is recorded in an execution trace with agent name, duration (ms), key finding, thought chain, and recommendation count.

3.4 Deterministic Conflict Resolution

Opposing recommendations trigger explicit arbitration using three trade-off rules:

- Performance vs. Cost (35% <= CPU < 75%): right-size paired with predictive autoscaling, 60-second scale-out buffer. Confidence: 0.88.
- Reliability vs. Cost (CPU < 35%): downsize while preserving automated horizontal failover. Confidence: 0.92.
- Carbon Deferral vs. Execution: non-critical batches shifted to green windows; interactive APIs remain on immediate execution paths. Confidence: 0.87.

Each resolution emits a ConflictResolutionItem record documenting conflicting agents, competing proposals, reconciliation rationale, final decision label, and confidence score. This audit trail supports explainability requirements in regulated cloud environments.

3.5 Non-Destructive Digital Twin Simulator

The Digital Twin engine evaluates candidate actions against a StateSnapshot without modifying live infrastructure. Five optimization scenarios: RIGHT_SIZE (projects downsized capacity on modern instance families), SCALE_UP, SCALE_DOWN (horizontal replica adjustments), CONSOLIDATE (bin-packing low-utilization workloads), REGION_MIGRATION (e.g., in-north ~650 gCO2/kWh to ca-central ~90 gCO2/kWh).

The engine computes baseline and projected Optimization Scores (0-100) via a composite scoring function across CPU efficiency, cost, carbon, and instance count. All outputs carry is_simulated=True and the label 'SIMULATED / ESTIMATED'.

3.6 AI Copilot and Safe Remediation Boundary

Eight grounded tools interface with live application state: get_cloud_metrics, get_history, get_predictions, get_cost_analysis, get_carbon_analysis, get_recommendations, get_cloud_health, and run_digital_twin_simulation. The copilot uses deterministic keyword/pattern matching with optional Gemini/OpenAI fallback. Non-dry-run remediation and rollback endpoints return HTTP 501 Not Implemented. Dry-run mode persists simulated audit events without provider API mutations.

4. Forecasting Methodology and Experimental Setup

4.1 Synthetic Dataset Generation

The synthetic generator produces 5,760 observations at 15-minute intervals from 2026-06-01 (seed 42). CPU = clip(35 + 30*sin^2((h_t - 7)/24 * 2*pi) * W_t + tau_t + epsilon_t, 2, 98), where W_t=0.7 on weekends, tau_t in [0,5]% linear trend, epsilon_t ~ N(0,16). Memory adds 0-8% upward drift (simulated memory leak). Network: clip(300 + 400*sin^2((h-9)/24*2*pi)*W_t + N(0,2500), 50, 2000) Mbps. Cost: clip(0.192*(1+cpu/100*0.5) + N(0,0.0001), 0.05, 5.0) USD/h [m5.xlarge baseline].

4.2 Carbon Target Definition and Dimensional Clarity

Regional grid intensity I(h_t) for us-east ranges 320-470 gCO2/kWh. Six regions: ca-central (79-120), eu-west (198-285), us-west (172-310), ap-southeast (460-565), in-north (585-740) gCO2/kWh. Assuming P=0.35 kW and dt=0.25 h: C_t = clip(I(h_t)*P*dt + N(0,25), 0, 500) [gCO2/15-min]. This excludes dynamic PUE, server power states, and Scope 3 emissions [4, 5].

4.3 Chronological Split and Target Construction

Targets are shifted forward delta=4 steps (future_col = df[col].shift(-4)). Trailing 4 rows without forward labels are dropped, yielding 5,756 complete records. An 80/20 chronological partition (no shuffling): training set - first 4,604 observations (48 days, 2026-06-01 to 2026-07-18); test set - final 1,152 observations (12 days, 2026-07-19 to 2026-07-30). Pandas rolling(4) is strictly backward-looking; no future information enters feature vectors.

4.4 Baseline Predictors and Evaluation Metrics

1. Persistence Baseline (y-hat_{t+4} = y_t): forecasts using the metric value at forecast origin t, correctly evaluating the 1-hour horizon.

2. Rolling Mean Baseline: evaluate_rolling_mean in baseline.py computes y_true=arr[window:], y_pred=mean(arr[i:i+window]). When called with the unshifted test series (not y_test), this evaluates y-hat_{t+1}=mu_{t,4} on the unshifted array (N=1,148) - a 15-min prediction. The stored rolling_mae reflects this 1-step horizon. The true four-step rolling baseline from origin t yields higher error than persistence for CPU (7.263% vs. 6.500%), memory, and network due to diurnal lag-induced phase shift.

Evaluation metrics: MAE = (1/N)*sum(|y_i - y-hat_i|); RMSE = sqrt((1/N)*sum((y_i - y-hat_i)^2)); R2 = 1 - sum((y_i - y-hat_i)^2) / sum((y_i - y-bar)^2); Gain(%) = 100 * (1 - MAE_model / MAE_persistence).

5. Experimental Results

5.1 Held-Out Forecasting Performance

Table 1 presents verified held-out metrics, reproduced identically by executing train.py with seed 42 (wall-clock: 6.4 seconds). All stored values in metrics.json were confirmed.

Table 1. *Verified Forecasting Metrics on Chronological Test Set (N=1,152). Persistence: y-hat_{t+4}=y_t. Stored Roll-4: repository 1-step moving average on unshifted data (N=1,148). True Roll-4: four-step rolling mean from forecast origin t.*

Target	MAE	RMSE	R2	Persist. MAE	Stored Roll-4	True Roll-4	Gain (%)
CPU Util. (%)	3.809	4.811	0.855	6.500	4.622	7.263	41.4
Memory Util. (%)	2.979	3.704	0.636	3.996	3.025	4.153	25.4
Net Traffic (Mbps)	43.876	55.328	0.871	80.346	56.727	90.816	45.4
Hourly Cost ($/h)	0.009	0.011	0.474	0.013	0.010	N/A	29.7
Carbon (gCO2/15m)	4.061	5.087	0.391	5.846	4.571	N/A	30.5

Gradient boosted regressors outperform persistence on all five targets. CPU and network achieve R2=0.855 and 0.871, reflecting strong pattern extraction from the sinusoidal generator. MAE improvements: 25.4% (memory) to 45.4% (network). Cost (R2=0.474) and carbon (R2=0.391) show lower variance explained due to additive observation noise on narrow base value ranges.

5.2 Feature Importance Analysis

- CPU: current lag (69.4%), hour of day (22.3%), day of week (4.8%), rolling mean (3.3%).
- Memory: current value (45.5%), hour (35.7%), rolling mean (17.7%), rolling std (0.9%).
- Network: current value (67.7%), hour (26.0%), day of week (4.0%), rolling mean (2.1%).
- Cost: CPU (67.1%), hour (21.9%), day of week (5.3%), memory (3.1%), current cost (2.5%). CPU dominance mirrors the generator formula cost = 0.192*(1 + cpu/100*0.5).
- Carbon: grid intensity (71.1%), hour (20.0%), CPU (8.5%), day (0.5%). Grid intensity dominance mirrors C_t = I(h_t)*E + epsilon.

5.3 Baseline Alignment Audit

The repository's stored rolling_mae evaluates y-hat_{t+1}=mu_{t,4} on unshifted test observations. At 15-minute intervals with high lag-1 autocorrelation, this achieves low MAE by predicting only 15 minutes ahead (CPU: 4.622%, memory: 3.025%, network: 56.727 Mbps). When evaluated as a true four-step-ahead baseline from forecast origin t (predicting y_{t+4} using mu_{t,4}), error increases: CPU MAE rises from

4.622% to 7.263%, memory from 3.025% to 4.153%, network from 56.727 to 90.816 Mbps.

Under correct horizon alignment, the true rolling-mean baseline performs worse than simple persistence for all three metrics. When CPU rises from early-morning troughs (~35%) toward mid-morning peaks (~65%), the rolling mean of the past hour underestimates the current trend. Gradient boosted models, with explicit hour-of-day features, correctly anticipate diurnal cycles and outperform both baselines.

6. Discussion

6.1 What the Results Establish

The results establish that gradient boosted regressors with backward-looking lag features and cyclical temporal indicators reliably recover the diurnal and weekly patterns of the synthetic generator. The 25.4%-45.4% MAE improvements over persistence are reproducible: with seed 42 fixed, the generator produces identical series each run. The 99-test, 100% pass-rate test suite confirms architectural correctness of all five agents, eight copilot tools, and the Digital Twin simulator.

The multi-agent architecture correctly implements five independent domain analyses with deterministic conflict resolution. Each arbitration decision is fully documented with rationale, competing recommendations, and confidence score - aligned with cloud governance explainability requirements.

6.2 What the Results Do Not Establish

The synthetic generator is a closed algebraic system; recovering its patterns does not constitute generalization to real cloud workloads. Real hyperscale telemetry from Google Borg, Alibaba Cluster Data 2018, and Azure Functions traces exhibits heavy-tailed job distributions, sudden preemptions, non-stationary distribution shifts, complex multi-tenant interference, and high autocorrelation with episodic bursts not in the synthetic series [3].

The system cannot claim: (i) verified production cost savings (HTTP 501 prevents live mutations); (ii) measured Scope 2 carbon reductions (synthetic proxy excludes PUE, server power states, and Scope 3); (iii) generalization to multi-tenant cloud environments; (iv) validated end-to-end live cloud connector behavior.

7. Limitations and Threats to Validity

1. Data Provenance: all evaluations derive from a single synthetic generator configuration (seed 42, 60 days, us-east grid curve). Varying these changes numeric results.
2. Recursive Multi-Step Inconsistency: forecast_multi_step applies 5-min recursive steps using models trained on direct 1-hour targets, and feeds back carbon output (~30-40 gCO2/15-min) into carbon_intensity (trained on ~350-450 gCO2/kWh) - ~10x mismatch.
3. Unverified Live Connectors: AWS, Azure, and GCP connectors fall back to synthetic data in test environments; end-to-end live polling was not validated against real accounts.
4. Protected Cloud Mutation: all non-dry-run and rollback endpoints return HTTP 501. Digital Twin savings figures are static model projections, not realized outcomes.
5. Carbon Accounting Scope: the synthetic target assumes a constant 350 W server, omitting dynamic power states (C0-C6), PUE variances, cooling overhead, and Scope 3 emissions.

6. Test Set Distribution: the 12-day test window covers a single contiguous segment; no cross-validation across multiple disjoint windows or seasonal periods.

8. Future Work

- Real Trace Evaluation: benchmark forecasters against public open traces (Google Borg v2.1, Alibaba Cluster Data 2018, Azure Functions) to assess generalization under non-stationary real-world distributions.
- Horizon Alignment Refactoring: replace forecast_multi_step recursive inference with direct multi-horizon models outputting [t+1, t+2, ..., t+H] simultaneously, eliminating error accumulation and the carbon scale mismatch.
- Real-Time Grid Integration: replace the static hourly carbon lookup with live API feeds (Electricity Maps, WattTime) for real-time marginal emissions factors.
- Live Canary Validation: deploy against an isolated canary cloud subscription with real billing telemetry to measure actual pre/post-intervention cost and carbon deltas.
- Uncertainty Quantification: add prediction intervals using gradient boosted quantile regression or conformal prediction for risk-aware decision support.

9. Conclusion

GreenMind AI demonstrates a complete, scientifically audited prototype integrating five one-hour-ahead gradient boosted forecasters, five-domain multi-agent orchestration, deterministic conflict arbitration, non-destructive Digital Twin simulation, and an eight-tool explainable AI copilot for cloud GreenOps. On a 60-day synthetic benchmark, MAE improves 25.4%-45.4% over persistence. A thorough technical audit verified all metrics identically, corrected the baseline horizon alignment discrepancy, documented the recursive multi-step scale mismatch, and established clear scope boundaries.

The architecture is technically sound for decision-support use. Production validation requires evaluation on open cluster traces, real-time grid intensity integration, recursive horizon refactoring, and end-to-end testing with isolated canary cloud accounts. This paper establishes a rigorous, reproducible baseline and transparent scope for future extension.

References

[1] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, Oct. 2001. doi: 10.1214/aos/1013203451.

[2] C. Bergmeir, R. J. Hyndman, and B. Koo, "A note on the validity of cross-validation for evaluating autoregressive time series prediction," Computational Statistics & Data Analysis, vol. 120, pp. 70-83, Apr. 2018. doi: 10.1016/j.csda.2017.11.003.

[3] A. Rossi, A. Visentin, D. Carraro, S. Prestwich, and K. N. Brown, "Forecasting workload in cloud computing: Towards uncertainty-aware predictions and transfer learning," Cluster Computing, vol. 28, art. no. 258, 2025. doi: 10.1007/s10586-024-04933-2.

[4] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," Journal of Machine Learning Research, vol. 21, no. 248, pp. 1-43, 2020.

[5] L. Lannelongue, J. Grealey, and M. Inouye, "Green algorithms: Quantifying the carbon footprint of computation," Advanced Science, vol. 8, no. 12, art. no. 2100707, 2021. doi: 10.1002/advs.202100707.

[6] A. Radovanovic, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Donohoe, N. de M. L. Jacob, P. Douglis, and W. D. Weihl, "Carbon-aware computing for datacenters," IEEE Transactions on Power Systems, vol. 38, no. 2, pp. 1270-1280, Mar. 2023. doi: 10.1109/TPWRS.2022.3173250.

[7] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, "Workload prediction using ARIMA model and its impact on cloud applications QoS," IEEE Transactions on Cloud Computing, vol. 3, no. 4, pp. 449-458, Oct.-Dec. 2015. doi: 10.1109/TCC.2014.2350475.

[8] J. Dodge, T. Prewitt, R. T. Des Combes, E. Odmark, R. Schwartz, E. Strubell, A. S. Luccioni, N. A. Smith, N. DeCario, and W. Buchanan, "Measuring the carbon intensity of AI in cloud instances," in Proc. 2022 ACM FAccT, Seoul, 2022, pp. 1536-1546. doi: 10.1145/3531146.3533147.

[9] Pratham-stack-coder, "GreenMind-AI repository," GitHub, 2026. [Online]. Available: https://github.com/Pratham-stack-coder/GreenMind-AI.